DFS running time – $\Theta(V+E)$

↓

each edge is explored twice (once from each end)

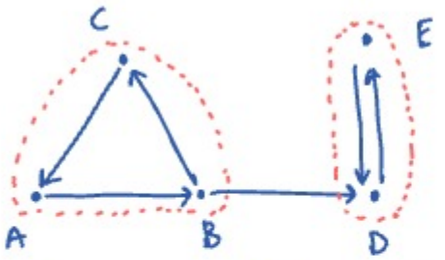
SCCs

BFS

```
explore(u):
    visited[u] = true
    for each neighbour v of u:
        if not visited[v]:
            explore(v)
```

```
dfs(G):
    for vertex u in G:
        if not visited[u]:
            explore(u)
            // components - count/mark etc
```

Connectivity in directed graphs:

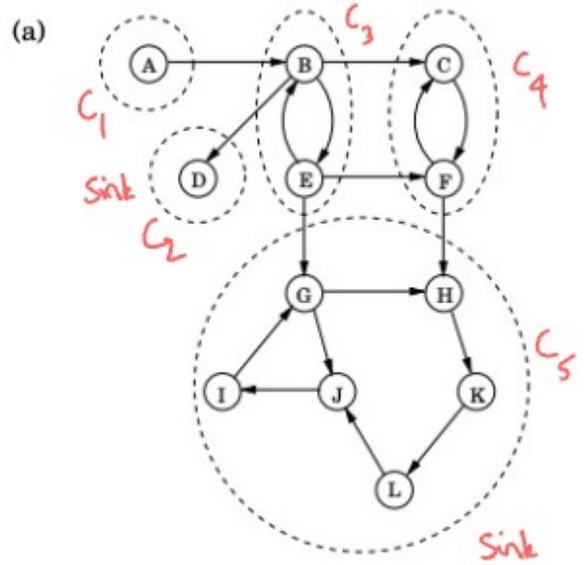


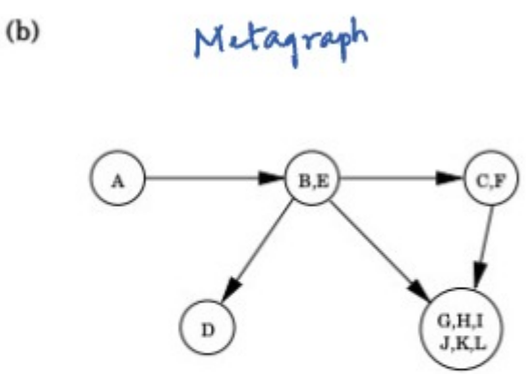
Metagraph is a DAG

Strong connectivity: In a strongly connected component (SCC), every vertex is reachable from every other in the SCC.

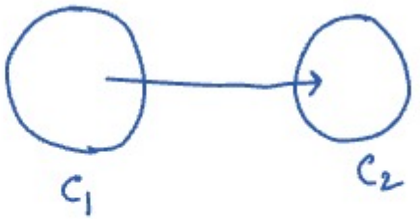
Eg: #SCCs in a DAG = #vertices

- Meta graph : Graph with vertices as SCCs

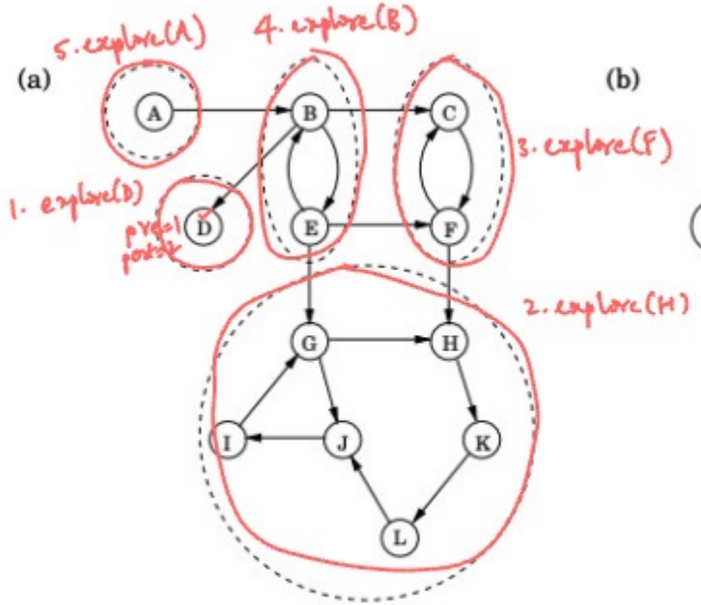


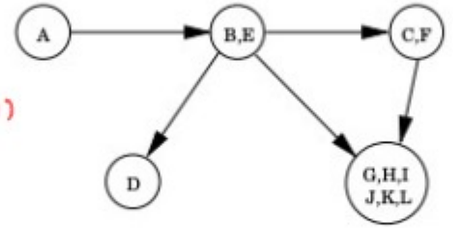


1. Running DFS starting from a vertex in a sink SCC will exactly discover all the nodes of that SCC.

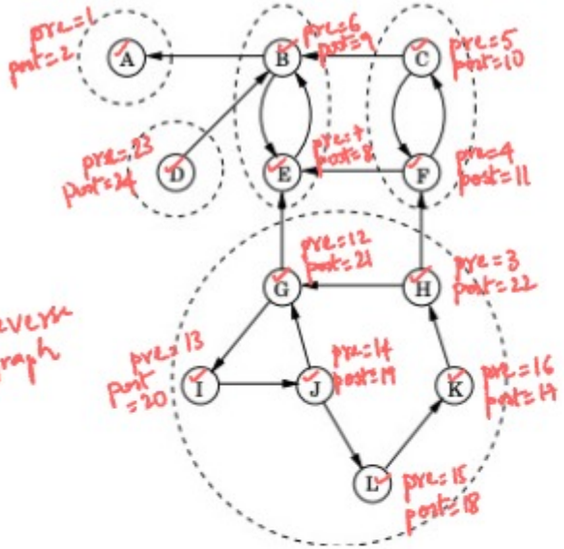


- The highest post # in C_1 > highest post # in C_2
- The vertex with the highest post # in the entire graph is part of a source SCC
- The vertex with the highest post # in the reverse graph is part of a sink SCC.

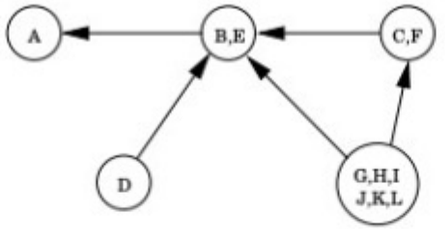




Kosaraju's two pass algorithm
$-\Theta(V+E)$

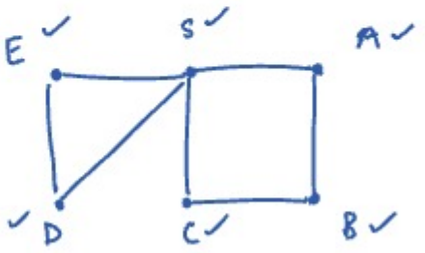


Reverse graph



Breadth First Search:

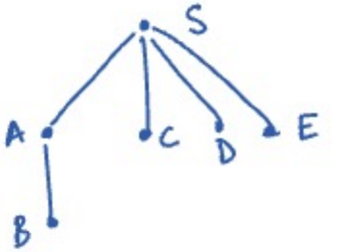
- Finds shortest paths from a source to ALL reachable vertices (in terms of # edges)

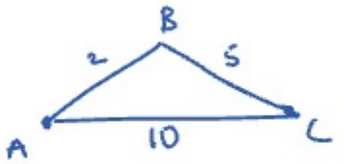


	S	A	B	C	D	E
prev =	[–	S	A	S	S	S]
visited =	[✓	✓	✓	✓	✓	✓]

queue

B E D C K S →

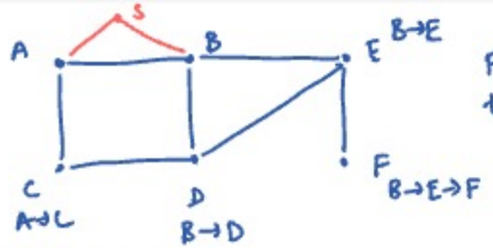




Shortest paths (weighted graphs)

- path length: sum of wts of edges along the path

- wts are non negative — Dijkstra's
- negative wts can be present — Bellman Ford's
- DAG — DP



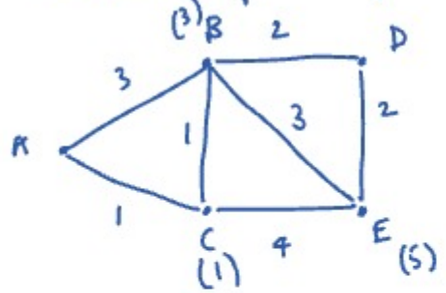
Find shortest path (#edges) to each vertex

has to be the min between shortest path len from A & shortest path len from B.

- say 'k' sources
- Brute force: run BFS from each of the k vertices $\Rightarrow \Theta(k(V+E))$

- Multi-source BFS
  - Add a super source 's' connecting to all the 'k' sources & run BFS from s
  - Practically, equivalent to adding all the 'k' sources to the queue initially in BFS.

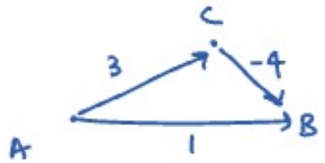
Dijkstra's Shortest path algorithm:



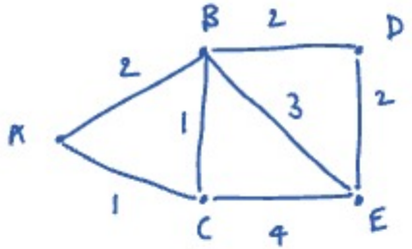
→

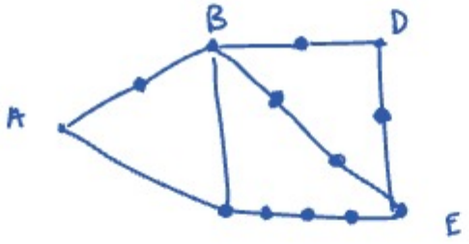
~~D(4) E(5) B(2) C(1) A(0)~~

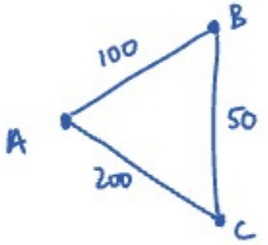
→A: 0
A→C: 1
C→B: 2
B→D: 4
C→E: 5



Negative weights







Correctness

vertices for whom the shortest paths are already found

distance(u) := Length of the shortest path from source to u

We selected a vertex w^* from the PQ s.t. $dist(v^*) + l_{v^*w^*}$ is the smallest d_1

Assume a path P' of shorter length than

$P : s \rightsquigarrow v^* \rightarrow w^*$

$P' : s \rightsquigarrow x \rightarrow y \rightsquigarrow w^*$

$dist(x)$ — non negative

$\Rightarrow len(P') < len(P)$

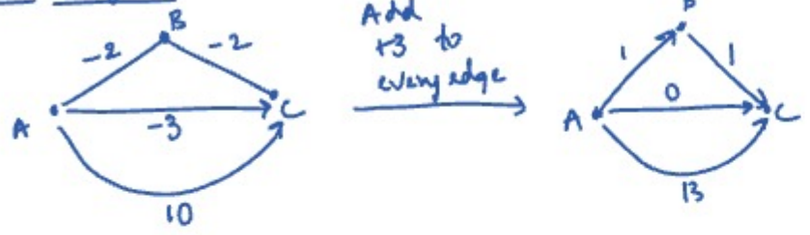
$dist(x) + l_{xy} +$ non-negative number $< dist(v^*) + l_{v^*w^*}$

$\Rightarrow dist(x) + l_{xy} < dist(v^*) + l_{v^*w^*}$

$\Rightarrow$ Contradiction

$\Rightarrow$ No such path P' exists.

Negative weights



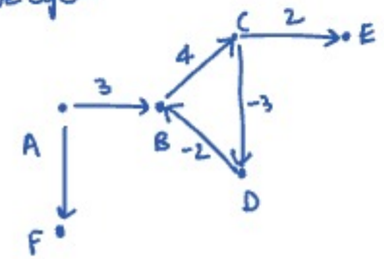
Add +3 to every edge →

$A \xrightarrow{-1} B \xrightarrow{-2} C$ (-4)

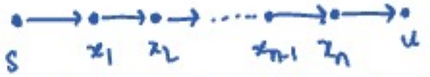
$A \to C$ (0)

Wrong reduction

- Negative cycles



- Bellman-Ford algorithm:
  - Consider a shortest path from s to u



Consider a point where, expense(x_n) = dist(x_n)

& if we relax(x_n, u),

that would set expense(u) to dist(u)

Relax operation of edge u→v

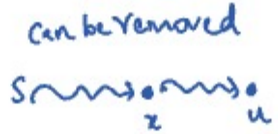
if expense(u) + w_{uv} < expense(v):

expense(v) = expense(u) + w_{uv}

Safe operation: it never updates the expense of a node to a value smaller than its actual distance

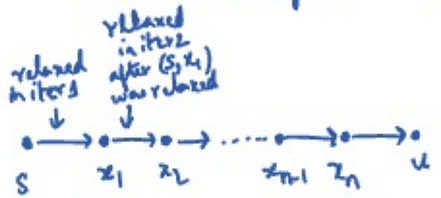
- In above shortest path, max #edges would be V-1

Can be removed



  - for V-1 times:

    relax all edges (in any order)



relax(s, x_1)
relax(x_1, x_2)
⋮
relax(x_{n-1}, x_n)
relax(x_n, u)

DP view of Bellman Ford's

DP solution:

$dp[i][v]$:= length of the shortest path from the source to vertex 'v' using atmost i edges

(distance of v from source)

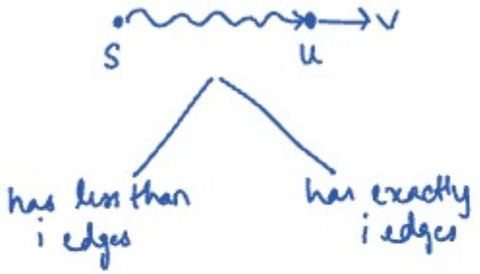
Base cases:

$dp[i][src] = 0$

$dp[0][v] = \infty$ ($v \neq src$)

Recurrence relation:

$$dp[i][v] = \min \left\{ dp[i-1][v], \min \{ dp[i-1][u] + w_{uv} \text{ for all } (u,v) \text{ in } G \} \right\}$$



$dp[j][u] \leq dp[k][u]$ if $j > k$

Order of filling:

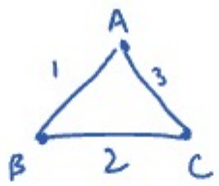
- Row by row & each row in any order (but preferably left to right for cache locality)

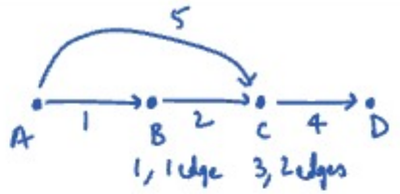
Final answer: $dp[V-1][x]$ would contain the distance to x from source

RT:

For each row, work done: $\sum_{v \in V} \text{indegree}(v) = E$

Total work: $\Theta(VE)$





A: [ ]
B: [(A, 1)]
C: [(B, 2), (A, 5)]
D: [(C, 4)]

Reverse Adj list

A: [(B,1), (C,5)]
B: [(C,2)]
C: [(D,4)]
D: [ ]

Normal Adj list

Optimizations:

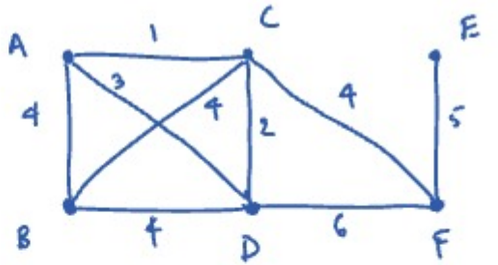
1. Convergence
2. Space optimization
3. Last row we only need $dp[k][dst]$
4. Top down is potentially faster
   - computes only what is required

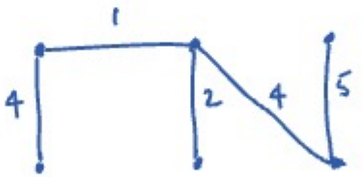
Minimum Spanning Trees: → has 'n' vertices

- Tree: A graph G is a tree iff

Equivalent definitions:
- G is connected & acyclic
- G is connected & has exactly 'n-1' edges
- G is acyclic & has exactly 'n-1' edges
- Every pair of vertices (u,v) is connected by a unique path
- G is minimally connected (if we remove one more edge, then it's not connected)
- G is maximally acyclic

- Spanning tree: A tree that connects (spans) all vertices of G
  - for such a tree to exist, G has to be connected
- We only consider undirected graphs.
- MST (Minimum Spanning Tree):
  - The spanning tree of G such that sum of weights of all edges is minimum.



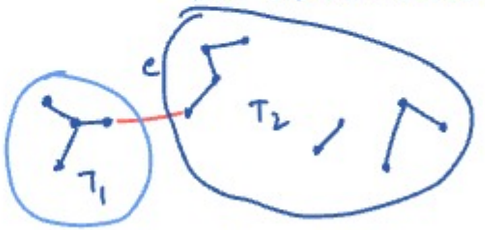


Kruskal's algo for MST: (edges in increasing order of weight)

for each edge in sorted edges:
  add edge if it doesn't create a cycle

↓ stop once we reach n-1 edges

- Kruskal's: each step we add the smallest edge that doesn't create a cycle



Consider $X = T_1$ $\quad V - X = V - T_1$

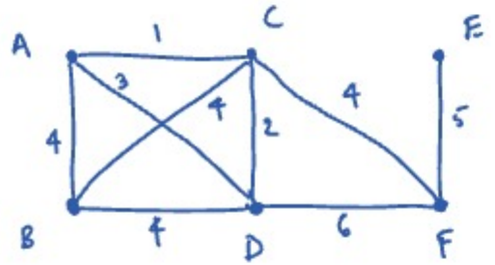
Min cut property: connected

- Given a graph $G(V, E)$ (undirected weighted graph)

For any partition of 2 subsets

$X, V-X$

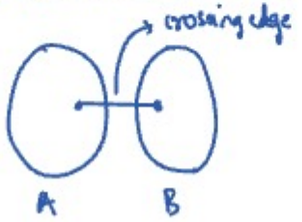
the crossing edge with the minimum weight has to be part of an MST.



Partition of a set into subsets (non-empty)

1. No overlap between any pair of subsets (pairwise disjoint or mutually exclusive)
2. All these subsets together should form the original set (collectively exhaustive)

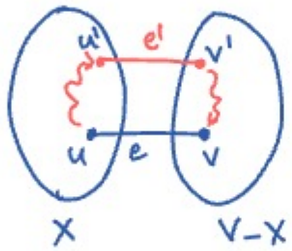
Crossing edge between 2 sets of vertices if edge has one end in one set & the other end in the other set



TODO:
- collectively exhaustive
- Bellman ford's case 2 DP
  dp[i-1][v]: min among shortest paths from s to v using exactly 'k' edges for k = 0...i-1
- Finish min cut property proof
- Prim's
- Disjoint sets

Proof

- Consider a partition $X, V-X$ of G
- Consider an MST 'T'
- 'e' is the crossing edge with min weight



Case1: e is part of T $\Rightarrow$ Done with proof

Case2: e is not part of T $\qquad wt(e) \leq wt(e')$

Adding e to T creates a cycle with e & e' in it

Removing e' from T is still a tree $= T'$ $\qquad wt(T') = wt(T) + wt(e) - wt(e')$

Prim's

T

crossing

Prim's: choose edge with the smallest weight

- We can start with any vertex as the source

Disjoint sets (Union find)

union(0,1) {0,1} {2},... ,{8}

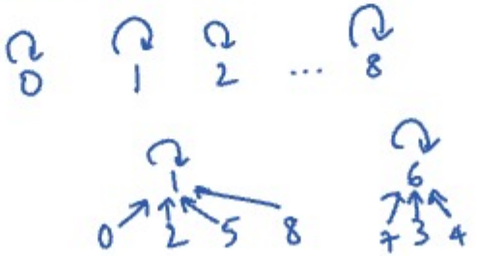
union(5,8)

union(2,8) {0,1} {2,5,8} {3},... {7}

union(1,5) {0,1,2,5,8} {3} {4} {6} {7}

find(5) → id of that set

- Quick find



	0	1	2	...	8
id	0	1	2	...	8
rank					

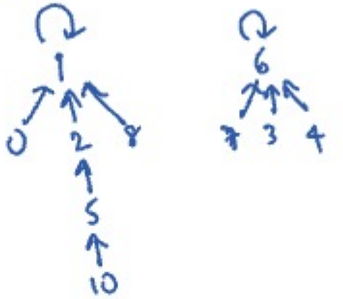
rank[] → store sizes of tree rooted at index i

weight/#nodes in tree

find - $\Theta(1)$

union - $O(n)$ operation as all ids of one set should be updated to root of other set

Weighted Quick union → if we attach smaller ht trees to largest ht trees ⇒ remains shallow



find - $O(\log n)$

union - $O(\log n)$

#atoms in the universe $\approx 10^{78} - 10^{82}$

- find(10) Path compression

(Weighted Quick Union with Path compression)

↳ Amortized RT: $O(\log^* n)$

$\log_2$ ⇒ # times we need to divide by 2 to get to 1

$\log^*_2$ ⇒ # times we need to apply log to get to 1

$2^{2} = 4$

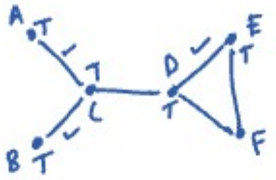
$2^{2^2} = 16$

$2^{2^{2^2}} = 2^4 = 2^{16} = 65536$

$2^{2^{2^{2^2}}} = 2^{65536} \approx 10^{20000}$

$\log^*(2^{2^{2^{2}}}) = 5$

```
ds = DisjointSets(n)
for edge in sorted edges:
    (u,v) = edge
    if ds.find(u) == ds.find(v):
        continue
    ds.union(u,v)
    mst_wt += edge.wt
```



- ✓ Graph - Medium + Hard
- ✓ Backtracking, Sorting

Word Search - II, Word ladder
Tree - 2 hard problems → Grid manipulation
LRU cache, LFU cache → 2048
Palindrome DP question → Train schedule

LC 918 - max subarray sum
Circular array
Heap last 4 problems (2 done)
Sliding window - 3 problems

Radix sort
Substring search

words = {hot, dot, dog, lot, log, cog}

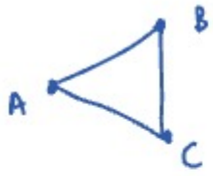
↓

{
- *ot : [hot, dot, lot]
- h*t : [hot]
- ho* : [hot]
- d*t : [dot]
- do* : [dot, dog]
- *og : [dog, log, cog]
- d*g : [dog]
- l*t : [lot]
- lo* : [lot, log]
- l*g : [log]
- c*g : [cog]
- co* : [cog]

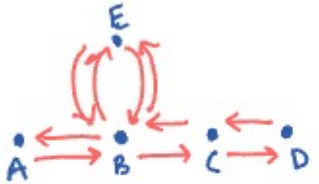
}

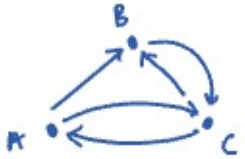
lot's neighbors { *ot, l*t, lo*

10^5 $10^{5/2}$



Euler's trail



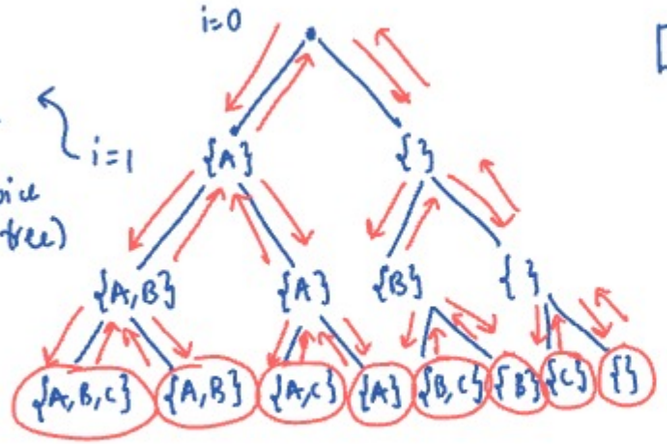



```
{
  A: [B, C]
  B: [C]
  C: [A, B]
}
```

Subsets

[A,B,C]

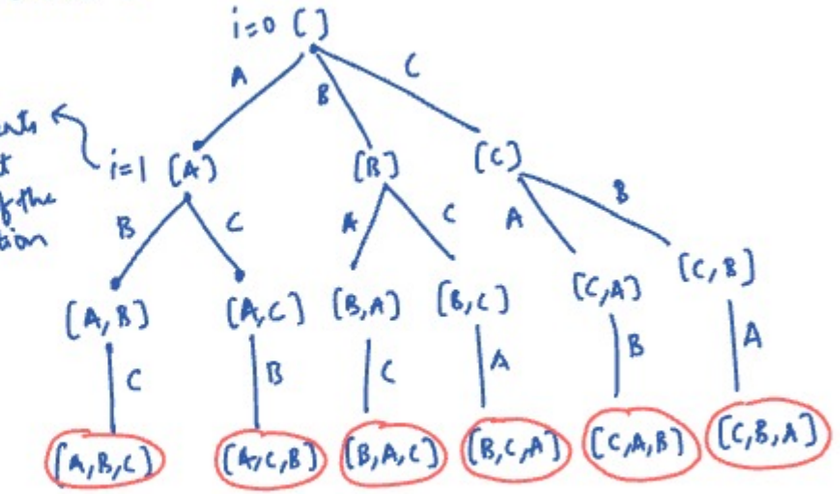
I.



Buffer

Subsets
A BC
AB
AC
A
BC
B
C
{}

#leaves = 2^n
#elements in tree = 2^{n+1}



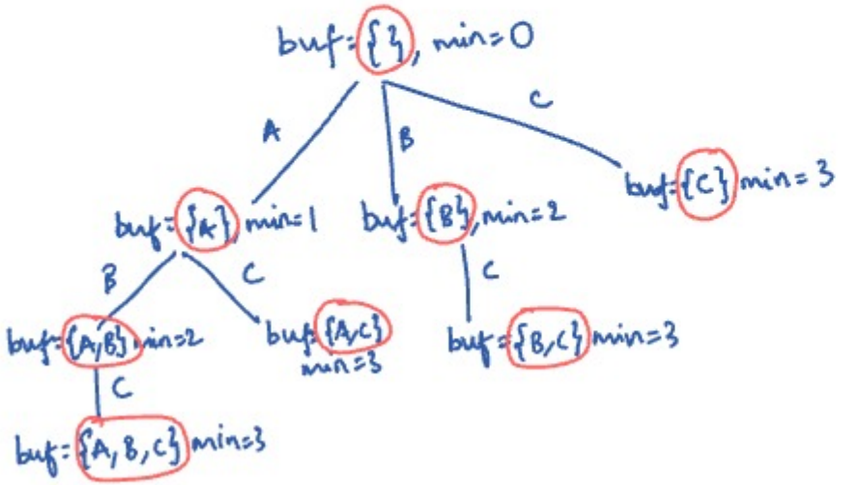
II. {A, B, C}

subsets formed from {B,C}: {{ } {B} {C} {B,C}} → recursively found

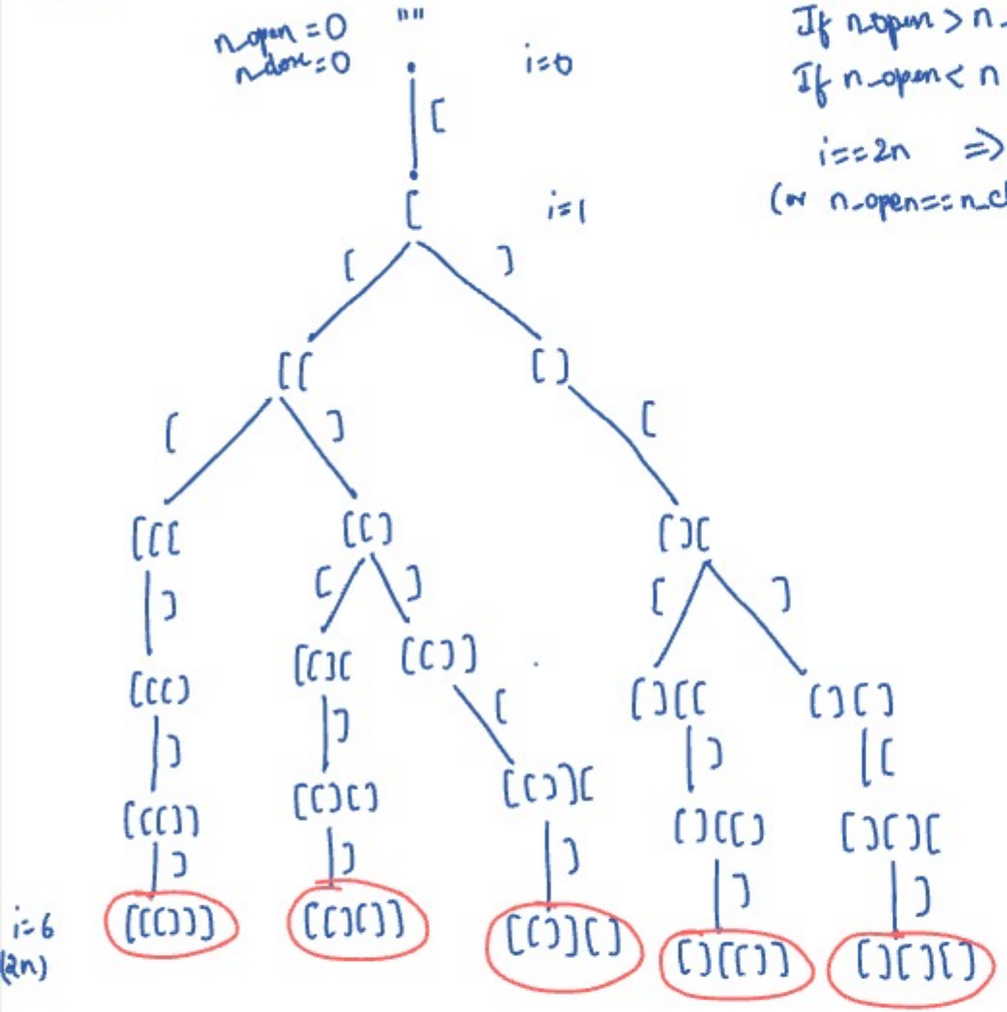
U {{A} {A,B} {A,C} {A,B,C}} → Add A to each of the subsets

[A,B,C]

III {A,C} or {C,A} ↔ [A,C]



n = 3



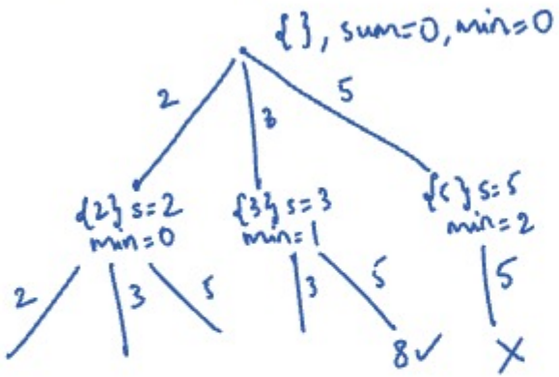
If n_open > n_close, we can add a closing

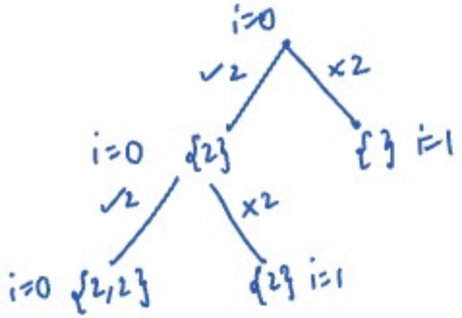
If n_open < n, we can add an opening

i == 2n ⇒ leaf (valid parenthesis string)

(or n_open == n_close == n)

nums = [2, 3, 5] target = 8






```
def f(..)
    if sum > target:
        return
```

- Sort the nums array

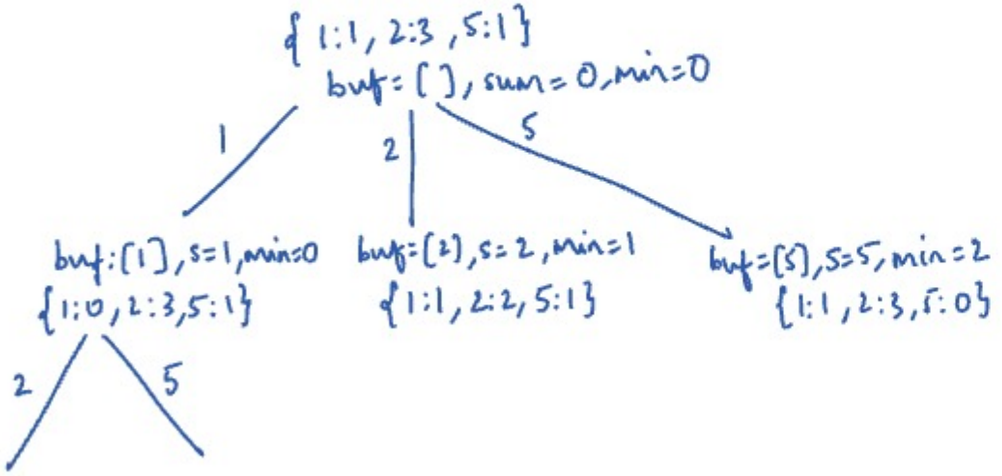
  sum + nums[i] ≥ target ⇒ sum + nums[j] > target ∀ j > i

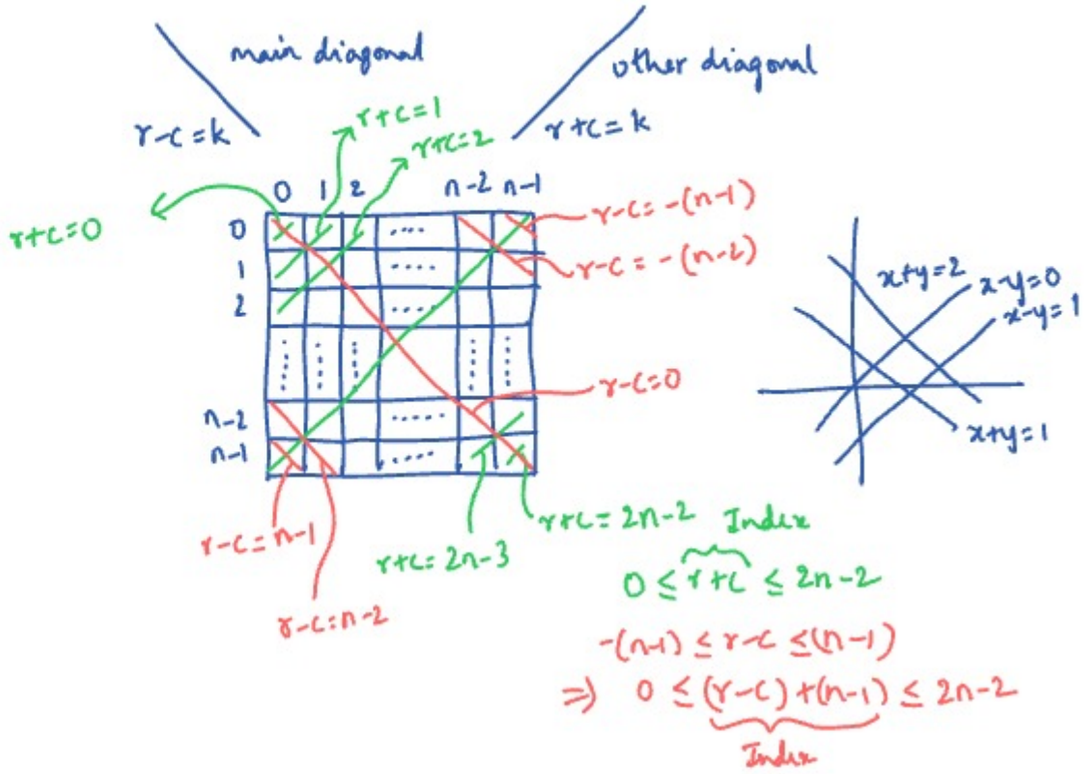
```
if sum + nums[i] > target:
```

nums = [2, 1, 5, 2, 2]

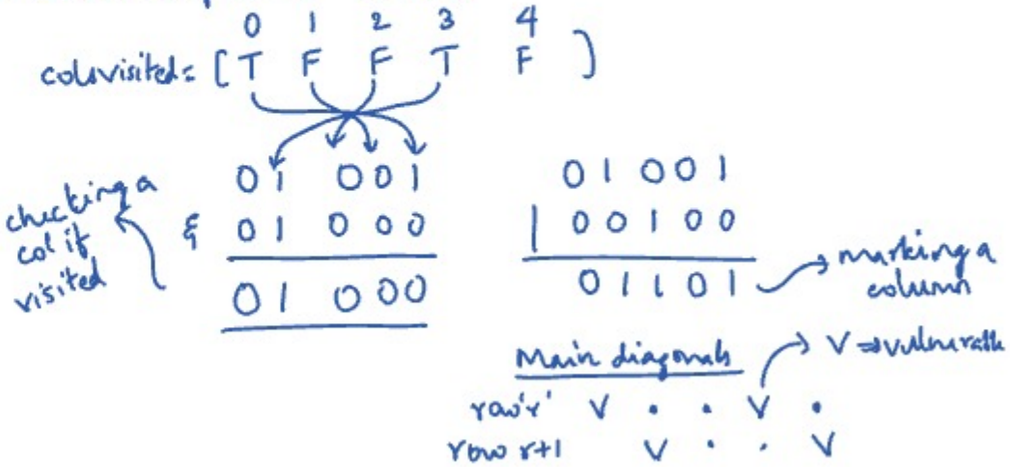
counts = [[1, 1], [2, 3], [5, 1]]

target = 5



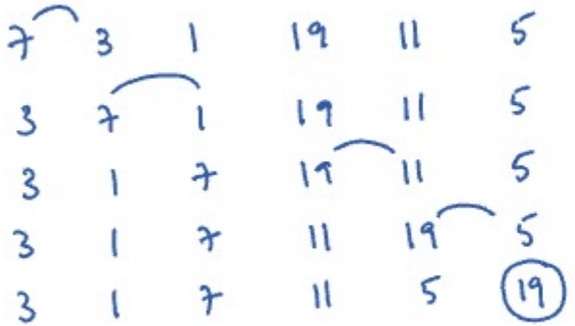


- Bitwise manipulation solution



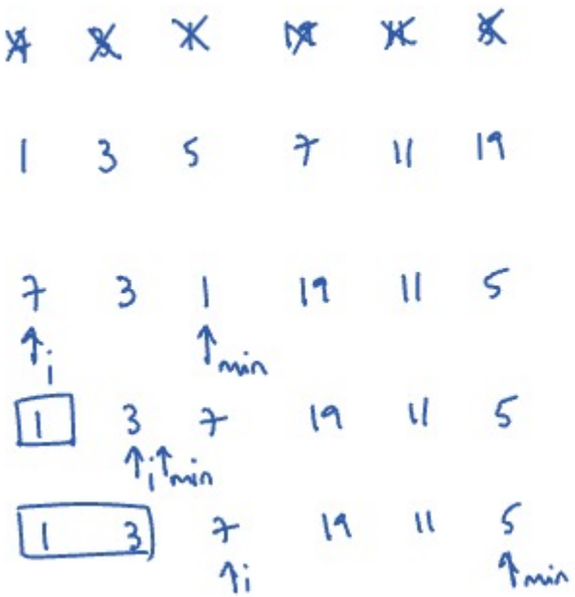
Sorting

- Bubble sort

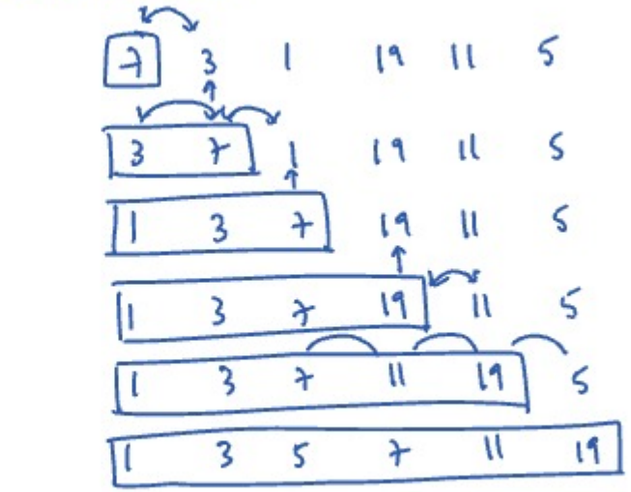


- After $n-1$ passes, array is sorted
- If no swaps occur in a pass $\Rightarrow$ array is sorted

Selection sort



Insertion sort:



We insert nums[i] into the sorted region.

4 7 9 10
3 7 9 10 4

3 steps per swap
3 swaps
$\Rightarrow$ 9 steps

	Best Case		Worst Case	
	# Compares	# swaps	# compares	# swaps
Bubble sort	$n-1$	0	$n(n-1)/2$	$n(n-1)/2$
Selection sort	$n(n-1)/2$	0	$n(n-1)/2$	$n-1$
Insertion sort	$n-1$	0	$\approx n(n-1)/2$	$\approx n(n-1)/2$

$(n-1)+(n-2)+\cdots+1$
$= \frac{n(n-1)}{2}$

- Inversions:
  - The pair of numbers that are out of order

7 3 1 19 11 5

(7,3) (3,1) (19,11) (11,5)
(7,1) (19,5)
(7,5)

- max #inversions in an array of size n
  $= (n-1)+(n-2)+\cdots+2+1$
  $= n(n-1)/2$

Partially sorted arrays: Arrays with
#inversions $\leq c \cdot n$
↓
$O(n)$

Insertion sort:

#swaps = #inversions

Work = #compares + #swaps
(#swaps + (n-1))
$= 2\,\#\text{swaps} + (n-1)$
$= 2\,\#\text{inversions} + (n-1)$

In each insertion
#compares = #swaps + 1

⇒ For partially sorted arrays, insertion sort runs in linear time.

Stability

- When duplicates are present in the array, the relative ordering among the duplicates being preserved is called stability

$A_1\ B\ A_2$

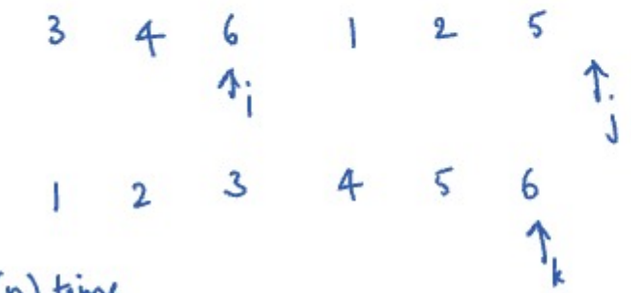
$A_1 A_2\ B$ → stable sort

- long distance swaps ⇒ Not stable
  ↓
  when it's not swapping adjacent elements

Bubble sort - stable
Insertion sort - stable
Selection sort - not stable
} inplace

- In place sorting algorithm:
  A sorting algo is inplace if extra space used is $\leq c \cdot \log n$ (auxillary)
  $O(\log n)$

- Merge sort:
  - sort left half & right half recursively
  - merge them



$\Theta(n)$ time
$\Theta(n)$ extra space

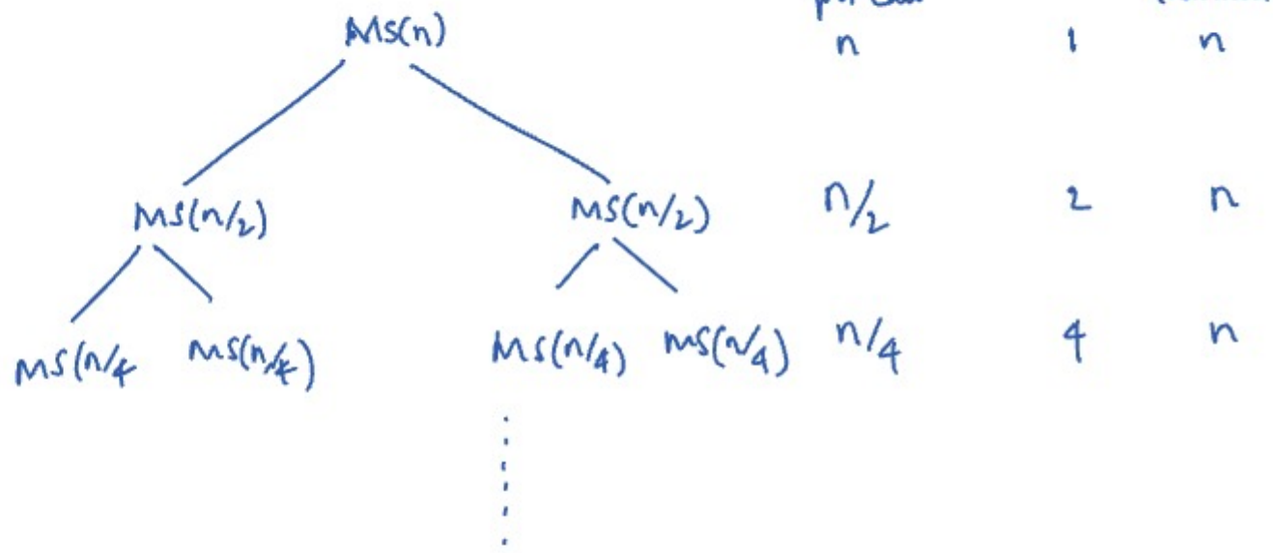
Choose element from left half always when comparing duplicates in merge step.

- $T(n)$:= running time for merge sort on an array of size n

$$T(n) = \underbrace{2T(n/2)}_{\text{merge step}} + \Theta(n)$$

$$T(n) = \Theta(n \log n)$$

$\frac{n}{2} \leq$ #compares $\leq n-1$
$\downarrow$
$\Theta(n)$



Total # levels = $\log_2 n$
Total work done = $n \log_2 n$